

宁波大学
NINGBO UNIVERSITY

实 验 报 告

[实验八 Linux 系统 Makefile 编写与 GCC 编译实验]

姓名：	赵瑞杰
学号：	216001944
课程名称：	嵌入式系统
所在学院：	信息科学与工程学院
专业班级：	21 级计算机科学与技术 1 班
任课教师：	王翀、王艺睿

实验项目名称	**Linux 系统 Makefile 编写与 GCC 编译实验**

一、实验目的与要求

1.1 实验目的

(1) 理解 Makefile 的作用和重要性，学习如何编写 Makefile 来管理复杂的编译过程。
(2) 掌握 Makefile 的基本语法和结构，能够编写 Makefile 来自动化编译任务。
(3) 掌握 GCC 编译器的使用方法，了解其基本选项和操作流程。
(4) 学习如何编译 C 语言程序，以及如何对编译过程进行控制和管理。

1.2 实验要求

(1) 学习 Makefile 中的目标（target）、依赖文件（dependency_file）和命令（command）的关系。
(2) 编写 Makefile 来编译一个简单的程序，包括设置编译器、编译选项和编译规则。
(3) 理解 Makefile 中的变量和模式规则，能够使用它们来简化 Makefile 的编写和维护。
(4) 准备实验环境，编写简单的 C 语言程序，并使用 GCC 编译器进行编译。
(5) 熟悉 GCC 编译过程中的各个阶段，包括预处理、编译、汇编和链接。

二、实验环境

(1) 华清远见开发环境。
(2) 实验平台为华清远见提供的虚拟机环境。
(3) 使用的操作系统是 Ubuntu 12.04 LTS 64-bit。
(4) 虚拟机工具软件为 VMware Player。
(5) FS4412 开发板。

三、实验过程、步骤及内容

3.1 Linux Shell 编程

将【华清远见-CORTEXA9 资料：\实验代码\2、Linux 移植驱动及应用\1、Linux 应用\实验代码\02. Linux 系统 shell 程序设计实验\实验代码】目录拷贝到虚拟机共享目录下。

建立相关目录：

```
$ mkdir workdir/linux/
$ mkdir workdir/linux/application
$ cd workdir/linux/application
```

拷贝完后的目录内容如下图所示：

```
linux@ubuntu64-vm:~/workdir$ cd linux/application/
linux@ubuntu64-vm:~/workdir/linux/application$ ls
11-pthread  7-Makefile  8-io  9-process
linux@ubuntu64-vm:~/workdir/linux/application$ mkdir 5-shell
linux@ubuntu64-vm:~/workdir/linux/application$ ls
11-pthread  5-shell  7-Makefile  8-io  9-process
linux@ubuntu64-vm:~/workdir/linux/application$
```

```
$ mkdir 5-shell
```

将代码从共享目录拷入虚拟机 Linux 操作系统下；

```
$ mkdir 5-shell
$ cp /mnt/hgfs/share/实验代码/02.\ Linux 系统 shell 程序设计实验/实验代码/* 5-shell/ -a
$ cd 5-shell/
```

拷贝完的 5-shell 目录下内容如下所示：

```
linux@ubuntu64-vm:~$ cd ~/workdir/linux/application/
linux@ubuntu64-vm:~/workdir/linux/application$ ls
11-pthread  5-shell  7-Makefile  8-io  9-process
linux@ubuntu64-vm:~/workdir/linux/application$ cp /mnt/hgfs/share/5/* 5-shell -a
linux@ubuntu64-vm:~/workdir/linux/application$ ls
11-pthread  5-shell  7-Makefile  8-io  9-process
linux@ubuntu64-vm:~/workdir/linux/application$ cd 5-shell/
linux@ubuntu64-vm:~/workdir/linux/application/5-shell$ ls
case_and_if.sh  file_or_dir.sh  simple  unload.sh
linux@ubuntu64-vm:~/workdir/linux/application/5-shell$
```

执行代码：

```
$ chmod  777  file_or_dir.sh                    //给脚本添加可执行权限
$ ./file_or_dir.sh                    //执行脚本，脚本的作用是判断该目录下的文件时目录还是普通文件
```

对应的代码如下所示：

```
#!/bin/sh
for i in *
do
    if [ -f $i ]
    then
        echo "$i is a file"
    elif [ -d $i ]
    then
        echo "$i is a directory"
    fi
done
```

3.2 GCC 编译

将【华清远见-CORTEXA9 资料：实验代码\2、Linux 移植驱动及应用\1、Linux 应用\实验代码\03. Linux 系统 GCC 编译器的使用实验\实验代码】目录拷贝到虚拟机共享目录下。

将 gcc 部分代码拷贝到虚拟机 Linux 下。

1、 建立相关目录；

```
$ cd workdir/linux/application
$ mkdir 6-gcc
```

2、 将代码从共享目录拷入虚拟机 Linux 操作系统下；（可使用【crtl+空格】切换输入法）

```
$ cp /mnt/hgfs/share/实验代码/03.\ Linux 系统 GCC 编译器的使用实验/实验代码/* 6-gcc/ -a
$ cd 6-gcc/
```

拷贝完成后的内容如下图所示：

```
linux@ubuntu64-vm:~/workdir/linux/application$ cp /mnt/hgfs/share/
5/ 6/
linux@ubuntu64-vm:~/workdir/linux/application$ cp /mnt/hgfs/share/6/* 6-gcc/ -a
linux@ubuntu64-vm:~/workdir/linux/application$ cd
11-pthread/ 5-shell/    6-gcc/      7-Makefile/ 8-io/       9-process/
linux@ubuntu64-vm:~/workdir/linux/application$ cd
11-pthread/ 5-shell/    6-gcc/      7-Makefile/ 8-io/       9-process/
linux@ubuntu64-vm:~/workdir/linux/application$ cd 6-gcc/
linux@ubuntu64-vm:~/workdir/linux/application/6-gcc$ ls
hello  helloworld.c
linux@ubuntu64-vm:~/workdir/linux/application/6-gcc$
```

编译代码：

```
$ arm-none-linux-gnueabi-gcc helloworld.c -o hello

$ mkdir /source/rootfs/app

$ cp hello /source/rootfs/app/
```

代码内容如下：

```
#include <stdio.h>


int main (int argc, char **argv)
{
    printf("hello,world!\n");


    return 0;
}
```

编译完成结果：

```
linux@ubuntu64-vm:~/workdir/linux/application/6-gcc$ ls
hello  helloworld.c
linux@ubuntu64-vm:~/workdir/linux/application/6-gcc$ cp hello /source/rootfs/app
linux@ubuntu64-vm:~/workdir/linux/application/6-gcc$ arm-none-linux-gnueabi-gcc
helloworld.c -o hello
linux@ubuntu64-vm:~/workdir/linux/application/6-gcc$ mkdir /source/rootfs/app
mkdir: 无法创建目录"/source/rootfs/app": 文件已存在
linux@ubuntu64-vm:~/workdir/linux/application/6-gcc$ cp hello /source/rootfs/app
linux@ubuntu64-vm:~/workdir/linux/application/6-gcc$
```

连接开发板后，在开发板串口终端执行应用程序。

```
# cd /app

# ./hello
```

3.3 Linux 系统 Makefile 编写实验

将【华清远见-CORTEXA9 资料：实验代码\2、Linux 移植驱动及应用\1、Linux 应用\实验代码\05. Linux 系统 Makefile 编写实验/实验代码/5.2/makefileTest】目录拷贝到虚拟机共享目录下。

将 Makefile 部分代码拷贝到虚拟机 Linux 下。

1、 建立相关目录；

```
$ cd workdir/linux/application

$ mkdir 7-Makefile
```

2、 将代码从共享目录拷入虚拟机 Linux 操作系统下；

```
$ cp /mnt/hgfs/share/实验代码/05.\ Linux 系统 Makefile 编写实验/实验代码/5.2/mak
```

```
efileTest/* 7-Makefile/ -a

    $ cd 7-Makefile/
```

拷贝完成后的内容如下图所示：

```
linux@ubuntu64-vm:~/workdir/linux/application$ ls
11-pthread  5-shell  6-gcc  7-Makefile  8-io  9-process
linux@ubuntu64-vm:~/workdir/linux/application$ cd 7-Makefile/
linux@ubuntu64-vm:~/workdir/linux/application/7-Makefile$ ls
bin  f1  f2  include  main  Makefile  obj
linux@ubuntu64-vm:~/workdir/linux/application/7-Makefile$
```

进入 makefileTest 目录，执行 make。

```
    $ make CLEAN

    $ make
```

代码内容如下：

```
    CC = arm-none-linux-gnueabi-gcc
    SUBDIRS = f1 \
              f2 \
              main \
              obj

    OBJS = f1.o f2.o main.o
    BIN = myapp
    OBJS_DIR = obj
    BIN_DIR = bin
    export CC OBJS BIN OBJS_DIR BIN_DIR

    all : CHECK_DIR $(SUBDIRS)
        cp bin/myapp /source/rootfs/app
    CHECK_DIR :
        mkdir -p $(BIN_DIR)
    $(SUBDIRS) : ECHO
        make -C $@
    ECHO:
        @echo $(SUBDIRS)
        @echo begin compile
    CLEAN :
        @$(RM) $(OBJS_DIR)/*.o
        @rm -rf $(BIN_DIR)
```

执行结果如下图所示：

```
linux@ubuntu64-vm:~/workdir/linux/application/7-Makefile$ make
mkdir -p bin
f1 f2 main obj
begin compile
make -C f1
make[1]: 正在进入目录 `/home/linux/workdir/linux/application/7-Makefile/f1'
make[1]: “../obj/f1.o”是最新的。
make[1]:正在离开目录 `/home/linux/workdir/linux/application/7-Makefile/f1'
make -C f2
make[1]: 正在进入目录 `/home/linux/workdir/linux/application/7-Makefile/f2'
make[1]: “../obj/f2.o”是最新的。
make[1]:正在离开目录 `/home/linux/workdir/linux/application/7-Makefile/f2'
make -C main
make[1]: 正在进入目录 `/home/linux/workdir/linux/application/7-Makefile/main'
make[1]: “../obj/main.o”是最新的。
make[1]:正在离开目录 `/home/linux/workdir/linux/application/7-Makefile/main'
make -C obj
make[1]: 正在进入目录 `/home/linux/workdir/linux/application/7-Makefile/obj'
make[1]: “../bin/myapp”是最新的。
make[1]:正在离开目录 `/home/linux/workdir/linux/application/7-Makefile/obj'
cp bin/myapp /source/rootfs/app
linux@ubuntu64-vm:~/workdir/linux/application/7-Makefile$
```

通过 tftp 下载内核，nfs 挂载文件系统，启动开发板。

在开发板串口终端执行应用程序。

```
# cd app

# ./myapp
```

我们也可以用如下命令清除中间文件和目标文件，恢复 make 之前的状态，在虚拟机下输入：

```
$ make CLEAN
```

恢复结果如下图所示：

```
linux@ubuntu64-vm:~/workdir/linux/application/7-Makefile$ ls
bin  f1  f2  include  main  Makefile  obj
linux@ubuntu64-vm:~/workdir/linux/application/7-Makefile$ make CLEAN
linux@ubuntu64-vm:~/workdir/linux/application/7-Makefile$ ls
f1  f2  include  main  Makefile  obj
linux@ubuntu64-vm:~/workdir/linux/application/7-Makefile$
```

四、实验结果

4.1 Linux Shell 编程

执行对应的代码文件结果如下图所示：

```
linux@ubuntu64-vm:~/workdir/linux/application/5-shell$ chmod 777 file_or_dir.sh
linux@ubuntu64-vm:~/workdir/linux/application/5-shell$ ./file_or_dir.sh
case_and_if.sh is a file
file_or_dir.sh is a file
simple is a directory
unload.sh is a file
linux@ubuntu64-vm:~/workdir/linux/application/5-shell$
```

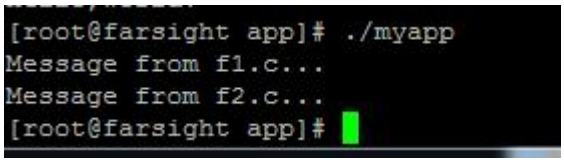
4.2 GCC 编译

在开发板上执行对应的应用程序：

```
[root@farsight ]# cd /app
[root@farsight app]# ls
hello
[root@farsight app]# ./hello
hello,world!
[root@farsight app]#
```

4.3 Linux 系统 Makefile 编写实验

在开发板上执行对应的应用程序：



五、实验结论

通过 Linux 系统 Makefile 编写与 GCC 编译实验，我深入理解了 Makefile 在管理复杂编译过程中的重要性，并学会了编写 Makefile 以自动化编译任务。通过实际操作，我掌握了 Makefile 的基本语法和结构，理解了目标、依赖文件和命令之间的关系。同时，通过编写和测试简单的 C 语言程序，我熟悉了 GCC 编译器的使用方法，了解了预处理、编译、汇编和链接等各个阶段的操作流程。这些知识和技能为我在 Linux 开发环境中更高效地管理和编译代码提供了基础。

在实验过程中，我将代码从共享目录复制到虚拟机 Linux 系统中，并成功编写和运行了 Makefile。通过执行 Makefile 命令，我实现了代码的编译和文件的自动化管理。此外，我通过 tftp 下载内核并使用 NFS 挂载文件系统，成功启动了开发板，并在开发板上执行了编译后的应用程序。整个实验过程不仅增强了我对 Linux 系统开发环境的理解，还提升了我的动手能力和解决实际问题的能力，为未来的嵌入式系统开发奠定了坚实的基础。